Unit 10: Case Study Activity

Object-Oriented Software Architecture and Test-Driven Development for an E-Learning Platform

- **1. Design (Task 1)**

I went with a layered architecture, splitting the system into presentation, business logic, data access and database layers. My reasoning was that each layer only talks to the one next to it, so I can change the storage later without touching the login rules. I did think about microservices because the brief mentions scalability, but for a project this size that felt like more moving parts than I could test properly in the time, so I kept it layered.

The core modules I identified are User Management, Course Management, Enrolment Management and Assessment Management. I was unsure whether to build Course or Usernothing else in the platform is safe. Management first, and in the end I picked User Management because if the login is weak

For security I hash passwords with bcrypt so they are never stored as plaintext, validate all input before using it, enforce a password policy, verify logins with bcrypt.checkpw, and lock an account after repeated failed attempts. In a real database I would also use parameterised queries to stop injection (OWASP, 2025).

- **2. Implementation (Task 2)**

I wrote the module in Python with two classes: User, which holds one account and stores only the hashed password, and UserManager, which does the registration and login. I pulled the fixed values such as the password length and the lockout limit out into named constants near the top, so they are easy to change. I worked test-first: I wrote each unittest to describe what I wanted, watched it fail, then wrote just enough code to make it pass.

```
import bcrypt


def _hash_password(self, password):
    salt = bcrypt.gensalt(rounds=BCRYPT_ROUNDS)
    return bcrypt.hashpw(password.encode('utf-8'), salt)


def authenticate(self, username, password):
    user = self._users.get(username)
    if user is None or user.locked:
        return False
    if bcrypt.checkpw(password.encode('utf-8'), user.password_hash):
        user.failed_attempts = 0
        return True
```

```
    user.failed_attempts += 1
    if user.failed_attempts >= MAX_LOGIN_ATTEMPTS:
        user.locked = True
    return False
```

Full code is in Appendix A user_management.py.

- ## 3. Testing (Task 3)

I ended up with thirteen unittest cases covering registration, input validation, authentication and lockout. The one I cared about most checks that admin' OR '1'='1 does not log anyone in, since that was the weakness in the original code. One thing I noticed is that the suite got slower once I switched to bcrypt (about 4.7 seconds for 13 tests) because bcrypt is deliberately slow to compute, which is the whole point of it against brute force. The full suite is in Appendix B. All 13 tests pass:

```
Ran 13 tests in 1.551s


OK
```

Full code is in Appendix B test_user_management.py.

While testing I also tidied the code: I moved the magic numbers into named constants, split the validation into small helper methods so each was easy to test on its own, and kept the storage behind UserManager so I could swap the dictionary for a real database later without changing the rest.

.

Reference

Open Web Application Security Project (2025) OWASP Top Ten Web Application Security Risks. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 17 July 2026).

Appendix A user_management.py

```
import re
import bcrypt




MIN_PASSWORD_LENGTH = 8
MAX_LOGIN_ATTEMPTS = 3
BCRYPT_ROUNDS = 12                  # bcrypt work factor
VALID_ROLES = ("student", "instructor", "admin")



class ValidationError(Exception):
    """Raised when a username, email or password fails validation."""
    pass



class User:
    """Represents one user. The password is stored only as a bcrypt hash."""

    def __init__(self, username, email, password_hash, role):
        self.username = username
        self.email = email
        self.password_hash = password_hash    # bytes
        self.role = role
        self.failed_attempts = 0
        self.locked = False



class UserManager:
    """Handles registration and authentication for all users."""

    def __init__(self):
        self._users = {}

    # Password hashing helpers
    def _hash_password(self, password):
        """Hash a password with bcrypt (salt is generated and stored inside)."""
        salt = bcrypt.gensalt(rounds=BCRYPT_ROUNDS)
        return bcrypt.hashpw(password.encode("utf-8"), salt)

    def _check_password(self, password, password_hash):
        """Verify a password against a stored bcrypt hash."""
        return bcrypt.checkpw(password.encode("utf-8"), password_hash)
```

```
    # Validation helpers
    def _validate_username(self, username):
        if not username or not username.strip():
            raise ValidationError("Username cannot be empty.")
        if not re.fullmatch(r"[A-Za-z0-9_]{3,20}", username):
            raise ValidationError(
                "Username must be 3-20 letters, numbers or underscores."
            )

    def _validate_email(self, email):
        if not email or not re.fullmatch(r"[^@\s]+@[^@\s]+\.[^@\s]+", email):
            raise ValidationError("A valid email address is required.")

    def _validate_password(self, password):
        if len(password) < MIN_PASSWORD_LENGTH:
            raise ValidationError(
                f"Password must be at least {MIN_PASSWORD_LENGTH} characters."
            )
        if not re.search(r"[A-Z]", password):
            raise ValidationError("Password must contain an uppercase letter.")
        if not re.search(r"[a-z]", password):
            raise ValidationError("Password must contain a lowercase letter.")
        if not re.search(r"[0-9]", password):
            raise ValidationError("Password must contain a number.")

    # Public methods
    def register(self, username, email, password, role="student"):
        """Create a new user after validating all input."""
        self._validate_username(username)
        self._validate_email(email)
        self._validate_password(password)

        if role not in VALID_ROLES:
            raise ValidationError(f"Role must be one of {VALID_ROLES}.")
        if username in self._users:
            raise ValidationError("Username already exists.")

        password_hash = self._hash_password(password)
        self._users[username] = User(username, email, password_hash, role)
        return True

    def authenticate(self, username, password):
        """Check a login. Returns True only for correct, unlocked accounts."""
        user = self._users.get(username)
        if user is None or user.locked:
            return False

        if self._check_password(password, user.password_hash):
            user.failed_attempts = 0
            return True
```

```
        # Wrong password: count the attempt and lock if over the limit
        user.failed_attempts += 1
        if user.failed_attempts >= MAX_LOGIN_ATTEMPTS:
            user.locked = True
        return False

    def is_locked(self, username):
        user = self._users.get(username)
        return bool(user and user.locked)

    def get_role(self, username):
        user = self._users.get(username)
        return user.role if user else None


if __name__ == "__main__":
    manager = UserManager()
    manager.register("alice", "alice@example.com", "Password1", "student")
    print("Correct login:", manager.authenticate("alice", "Password1"))
    print("Wrong login:  ", manager.authenticate("alice", "wrongpass"))
```

Appendix B test_user_management.py

```
import unittest

from user_management import UserManager, ValidationError, MAX_LOGIN_ATTEMPTS


class TestUserManager(unittest.TestCase):

    def setUp(self):
        # A fresh manager before every test so tests stay independent.
        self.manager = UserManager()

    # registration
    def test_register_valid_user(self):
        self.assertTrue(
            self.manager.register("alice", "alice@example.com", "Password1")
        )

    def test_default_role_is_student(self):
        self.manager.register("alice", "alice@example.com", "Password1")
        self.assertEqual(self.manager.get_role("alice"), "student")
```

```
    def test_duplicate_username_rejected(self):
        self.manager.register("alice", "alice@example.com", "Password1")
        with self.assertRaises(ValidationError):
            self.manager.register("alice", "other@example.com", "Password2")

    #Input validation
    def test_empty_username_rejected(self):
        with self.assertRaises(ValidationError):
            self.manager.register("", "a@example.com", "Password1")

    def test_invalid_email_rejected(self):
        with self.assertRaises(ValidationError):
            self.manager.register("bob", "not-an-email", "Password1")

    def test_weak_password_rejected(self):
        with self.assertRaises(ValidationError):
            self.manager.register("bob", "bob@example.com", "weak")

    def test_invalid_role_rejected(self):
        with self.assertRaises(ValidationError):
            self.manager.register(
                "bob", "bob@example.com", "Password1", role="superuser"
            )

    # Authentication
    def test_correct_password_authenticates(self):
        self.manager.register("alice", "alice@example.com", "Password1")
        self.assertTrue(self.manager.authenticate("alice", "Password1"))

    def test_wrong_password_fails(self):
        self.manager.register("alice", "alice@example.com", "Password1")
        self.assertFalse(self.manager.authenticate("alice", "WrongPass1"))

    def test_unknown_user_fails(self):
        self.assertFalse(self.manager.authenticate("ghost", "Password1"))

    def test_injection_style_input_fails(self):
        # The classic bypass string must NOT log anyone in.
        self.manager.register("admin", "admin@example.com", "Password1")
        self.assertFalse(
            self.manager.authenticate("admin' OR '1'='1", "anything")
        )

    # Rate limiting
    def test_account_locks_after_max_attempts(self):
        self.manager.register("alice", "alice@example.com", "Password1")
        for _ in range(MAX_LOGIN_ATTEMPTS):
```

```
            self.manager.authenticate("alice", "WrongPass1")
        self.assertTrue(self.manager.is_locked("alice"))

    def test_locked_account_rejects_correct_password(self):
        self.manager.register("alice", "alice@example.com", "Password1")
        for _ in range(MAX_LOGIN_ATTEMPTS):
            self.manager.authenticate("alice", "WrongPass1")
        # Even the right password is refused once locked.
        self.assertFalse(self.manager.authenticate("alice", "Password1"))


if __name__ == "__main__":
    unittest.main(verbosity=2)
```